Chandubhai S Patel Institute of Technology

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement: Containerization with Docker and Docker Compose.

Screenshots (Supplementary Problems are also implemented):

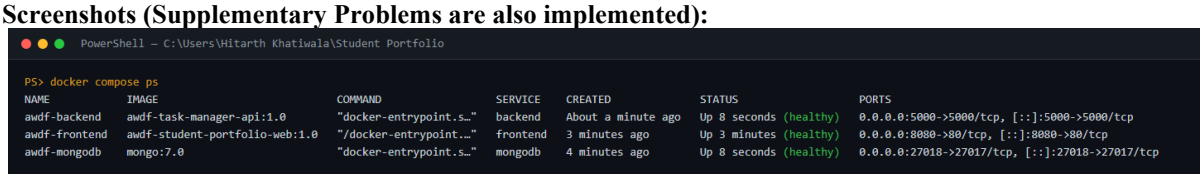


Figure 1: *Output of docker compose ps: the awdf-mongodb, awdf-backend and awdf-frontend containers are Up and (healthy), with ports 27018, 5000 and 8080 published to the host.*

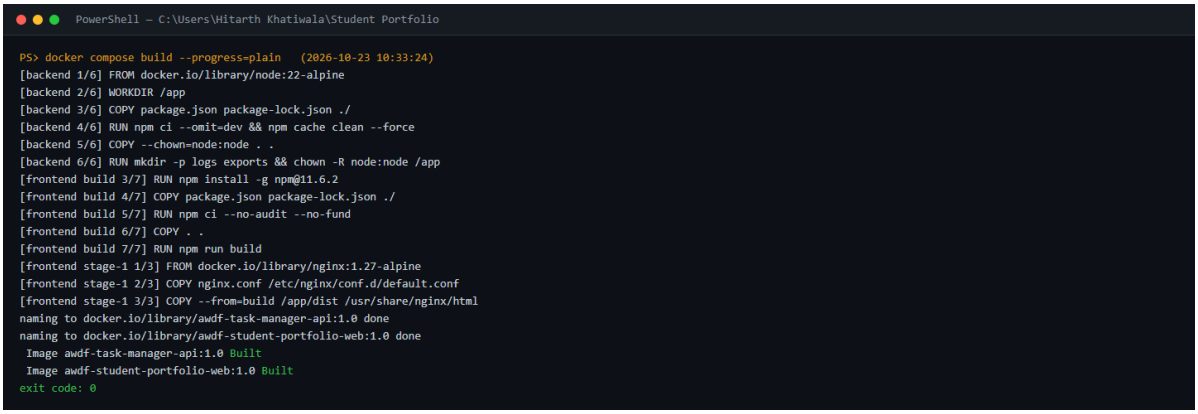


Figure 2: *docker compose build: the API image (node:22-alpine, npm ci --omit=dev, non-root) and the multi-stage frontend image (Vite build stage on Node, then only the static dist copied into nginx:1.27-alpine).*



Figure 3: *docker compose up -d: Compose creates the two networks and two named volumes, then starts the services in dependency order - the backend waits for MongoDB to be Healthy and the frontend waits for the backend.*

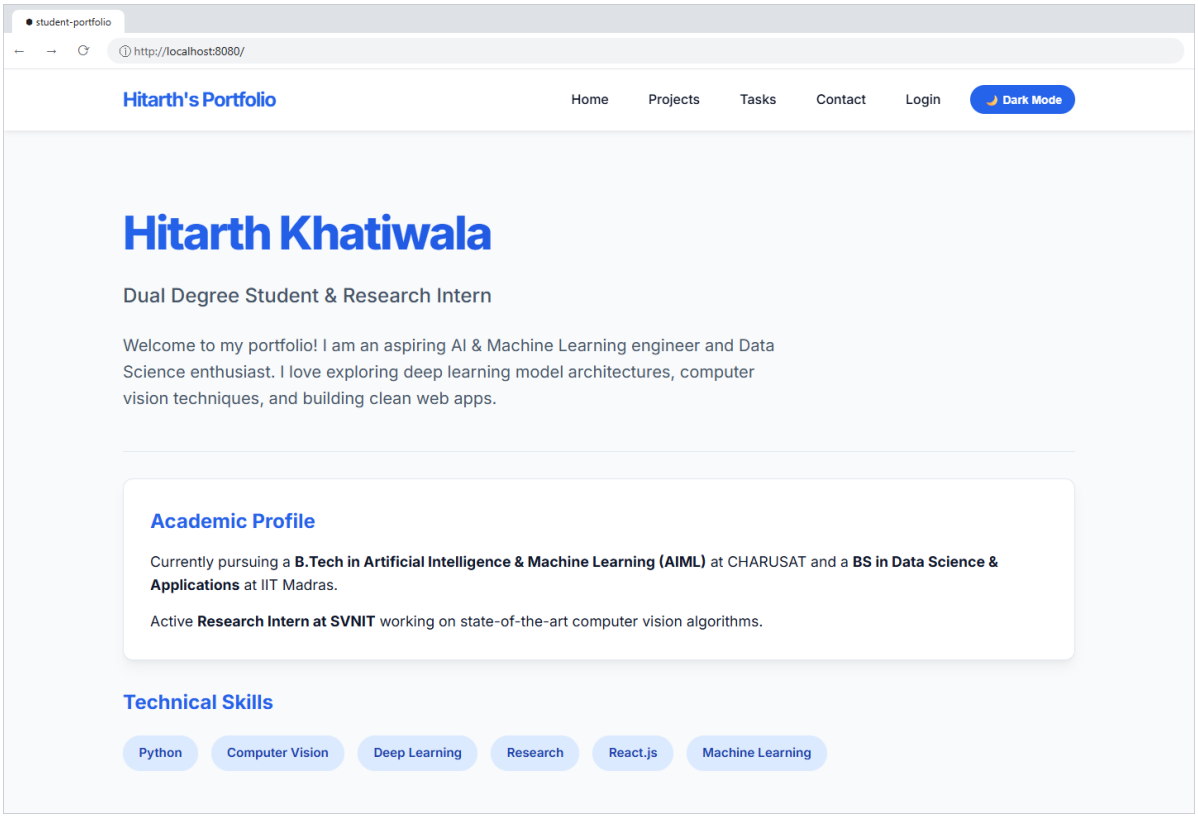


Figure 4: *Portfolio home page served by the containerized frontend (Nginx) on http://localhost:8080.*

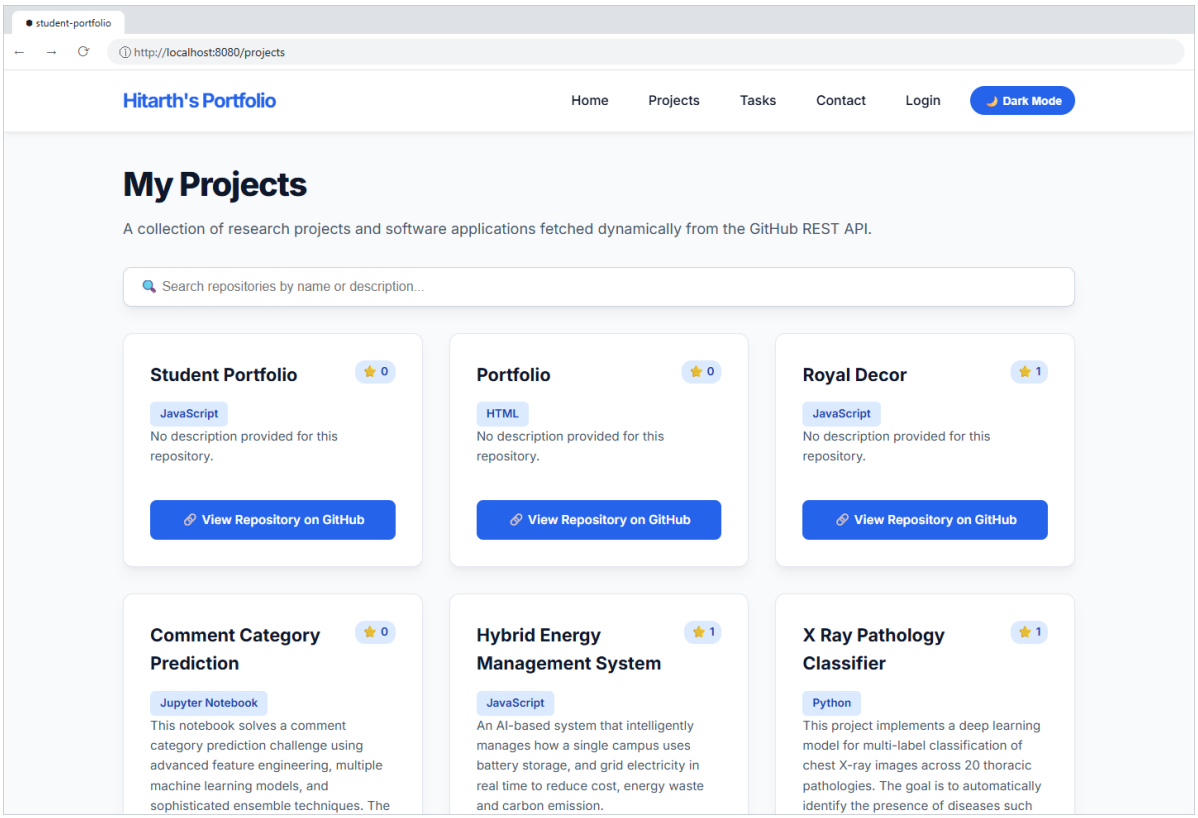


Figure 5: *Projects page on localhost:8080, confirming client-side routes work through the Nginx SPA fallback (try_files ... /index.html).*

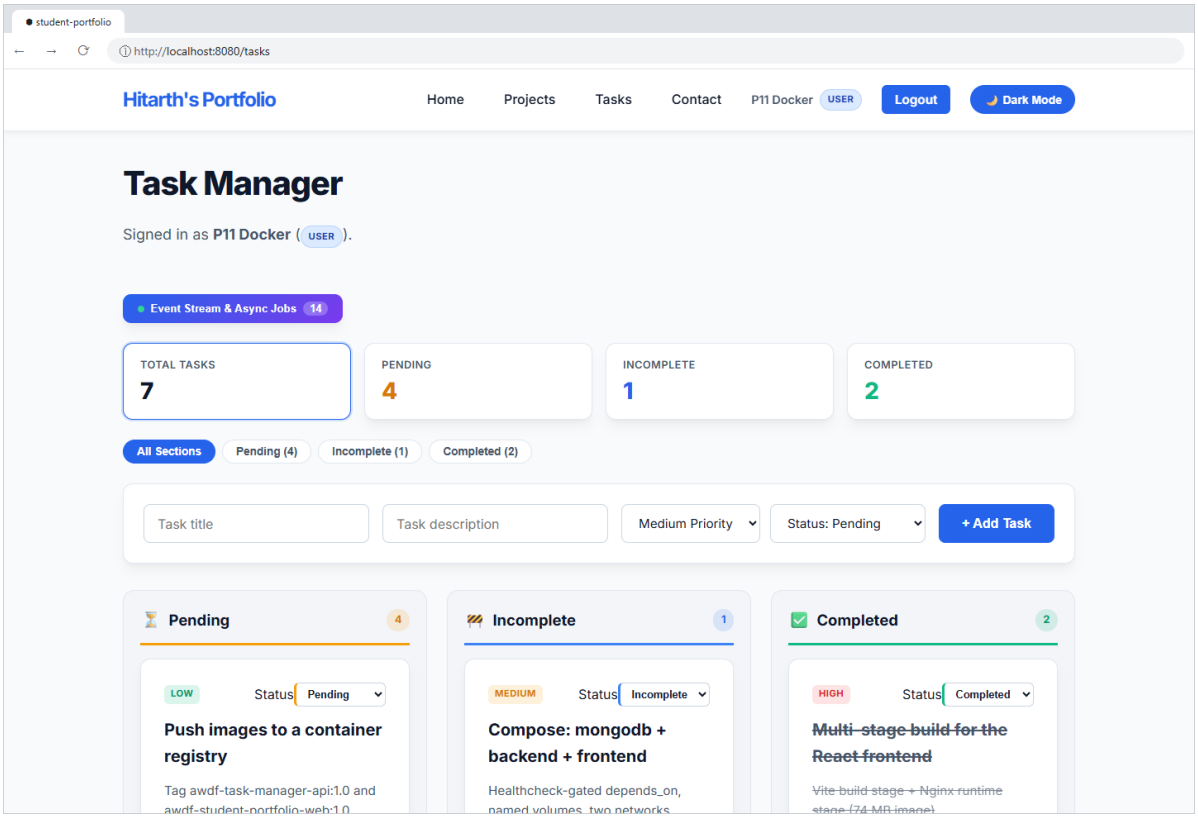


Figure 6: *Tasks page on localhost:8080: data is read from the MongoDB container through the Nginx reverse proxy (/backend/* -> http://backend:5000 on the Compose network).*

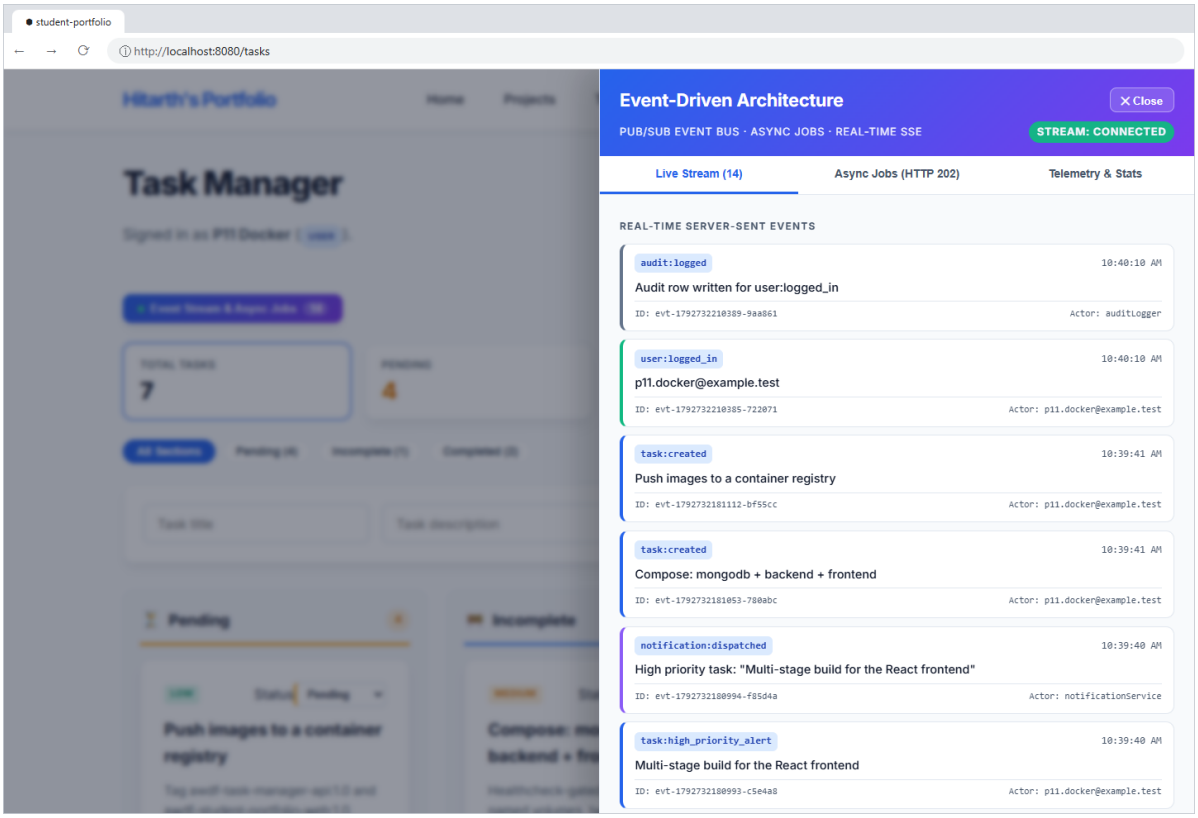


Figure 7: *Practical 10 event stream working inside Docker: the SSE connection stays open through Nginx (proxy_buffering off) and shows events published by the backend container.*

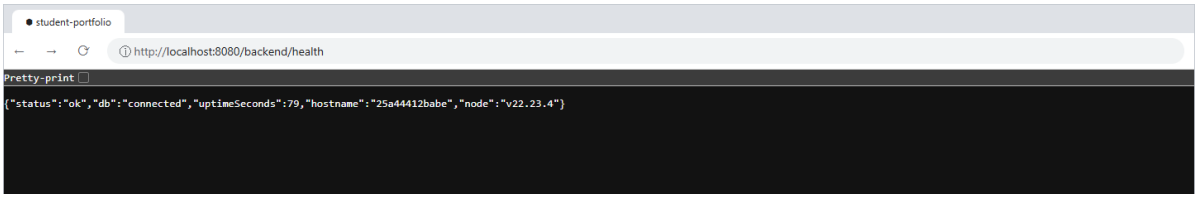


Figure 8: *GET /backend/health - the endpoint used by the backend HEALTHCHECK, reporting the MongoDB connection, uptime and the container hostname.*

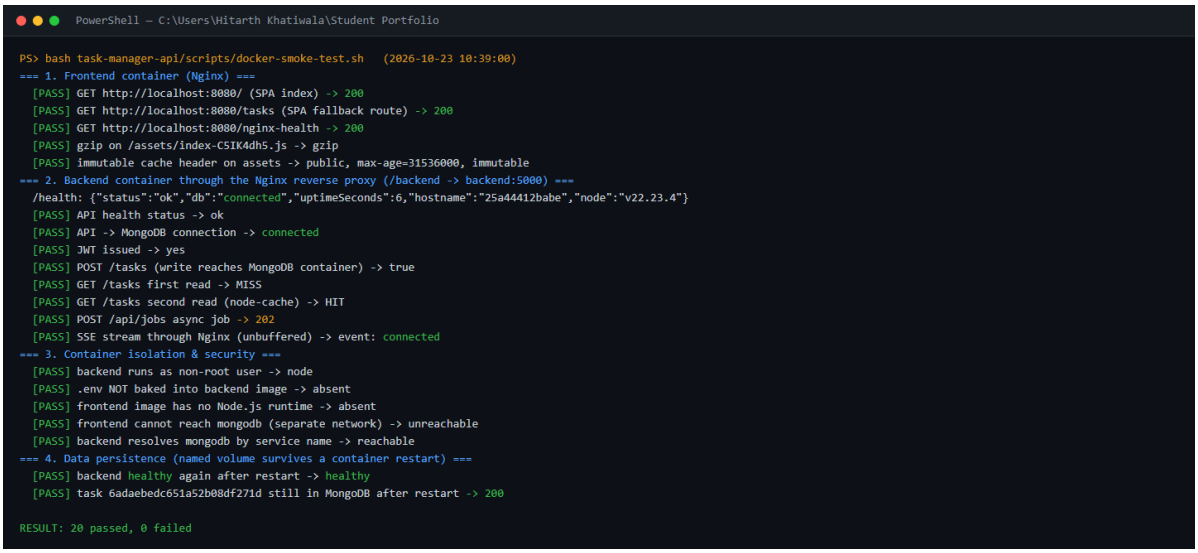


Figure 9: *Automated smoke test (scripts/docker-smoke-test.sh): 20/20 checks passed - Nginx SPA routing, gzip and cache headers, API via proxy, JWT, cache HIT/MISS, HTTP 202 jobs, SSE, non-root user, no .env in the image, network isolation and data persistence after a restart.*

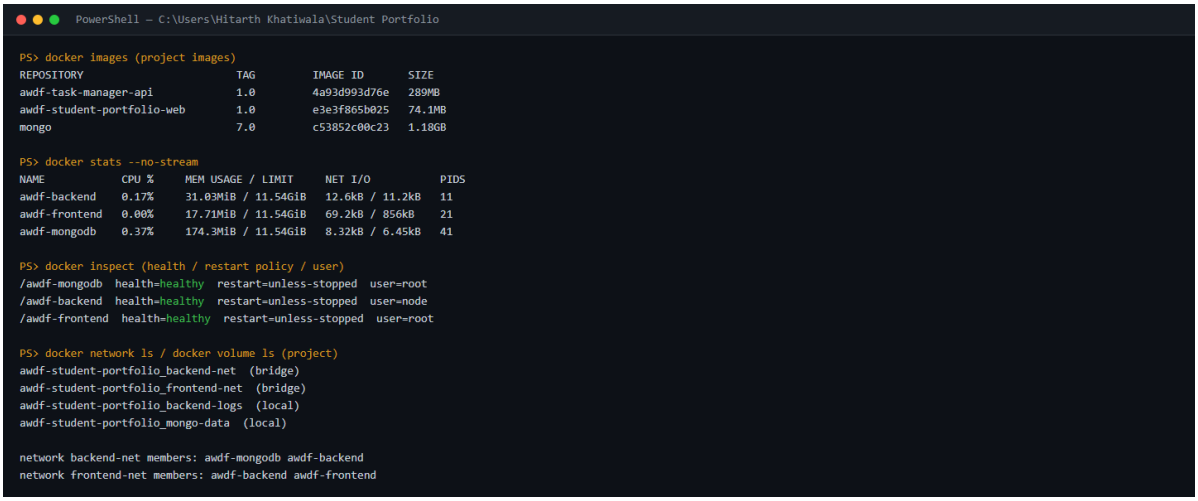


Figure 10: *Image sizes (frontend 74 MB vs API 289 MB), live resource usage from docker stats, health status and restart policy per container, and the two isolated networks: the frontend cannot reach MongoDB.*

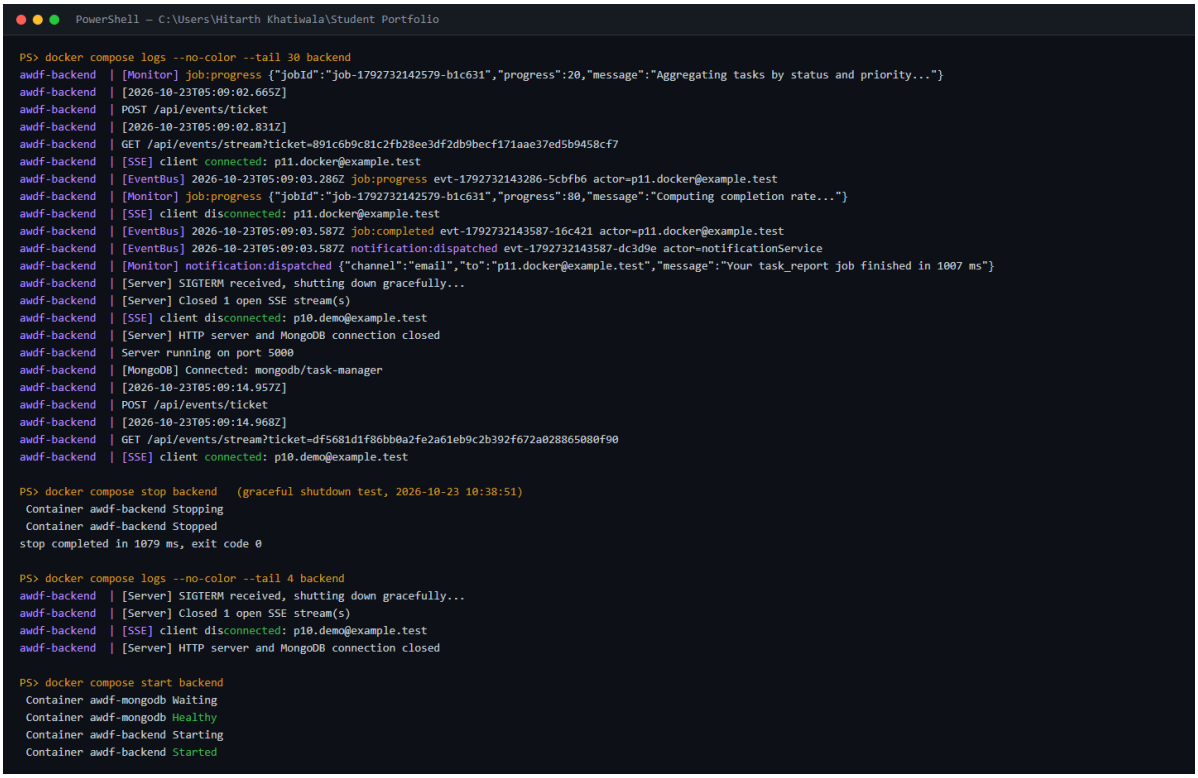


Figure 11: *docker compose logs backend plus a graceful shutdown test: on SIGTERM the API closes open SSE streams, the HTTP server and the MongoDB connection, and stops in about 1 s with exit code 0.*

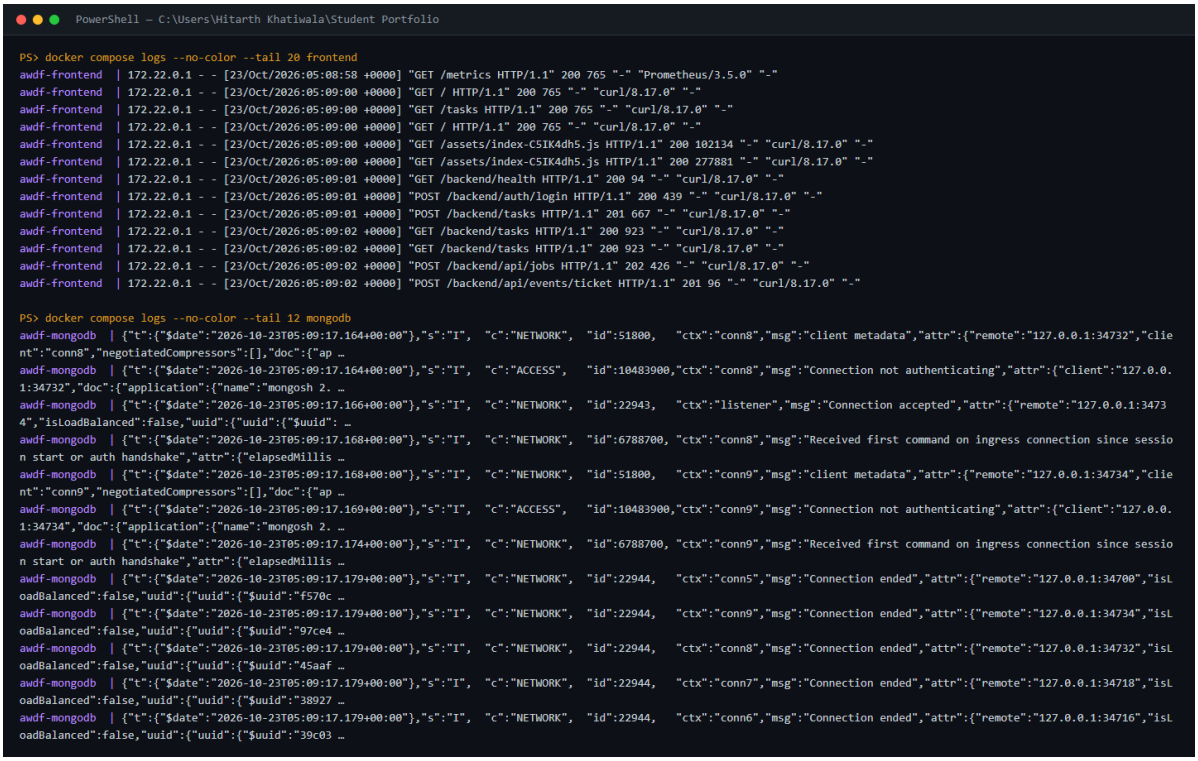


Figure 12: *docker compose logs for the frontend (Nginx access log of proxied API and asset requests) and MongoDB (connections accepted from the backend container).*

Github Repo Link: https://github.com/hitarth1812/Student-Portfolio